

PORTFOLIO · FULL-STACK WEB DEVELOPER

최성광

사용자 화면부터 서버·데이터 파이프라인까지, 서비스 전 구간을 직접 구축하는 개발자

sungkwang0908@naver.com | github.com/CHOISUNGKWANG/portfolio-2026 | Microsoft AI School 10기 수료

이약머약

사진 한 장으로 알약을 식별하는
AI 복약 안내 웹 서비스

프론트엔드 리드 · 서버 구축 · 데이터 관리/증강

채워줘, 홈즈!

빈방 사진에 가구를 자동 배치하는
AI 홈 스타일링 웹/앱 서비스

백엔드 서버 구축 · 핵심 AI 파이프라인 구현

이 포트폴리오는 이렇게 구성되어 있습니다

01

p.3 – 6

CAREER

(주)에스파이온코리아 · 3년 10개월

- CAMIS 의료정보시스템 운영·개발
- 반복 조회(N+1) 구조 개선
- 롤백을 전제로 한 배포·마이그레이션

02

p.8 – 14

PROJECT 01 이약머약

알약 이미지 인식 AI 서비스

- 프론트엔드 전체 담당 + 서버 구축
- 데이터 증강으로 실환경 인식을 개선
- Azure VM · FastAPI · Custom Vision

03

p.15 – 23

PROJECT 02 채워줘, 홈즈!

빈방 사진 기반 AI 홈 스타일링

- 백엔드 · 4단계 AI 파이프라인 단독 구현
- 생성형 AI 한계를 3D 실측 렌더링으로 돌파
- Gemini Robotics-ER · nvdiffrast

04

p.24 – 27

PROJECT 03 ForeShield

기후재난 6종 예측 시스템

- 6인 팀에서 대화 도메인 백엔드 전담
- 인증·동시성·스키마 문제를 찾아내 해결
- FastAPI · PostgreSQL · SSE

운영으로 검증된 신중함 위에, 직접 구축해본 경험과 팀으로 만든 경험을 엮었습니다 — 지키는 개발과 만드는 개발을 모두 담았습니다.

3년 10개월, 의료정보시스템(CAMIS) 운영·개발

CAMIS — 의료기관용 정보시스템

여러 의료기관이 사용하는 웹 기반 정보시스템의 유지보수·신규 개발을 담당. 진료 자연이 곧 업무 중단으로 이어지는 환경에서, 코드를 '동작하게'가 아니라 '안정적으로 운영되게' 만드는 데 집중했습니다.

담당 역할

- 웹 서비스 운영 및 신규 개발
- 장애 대응 · 배포 관리
- 신규 기관 온보딩 현장 파견

신규 의료기관 온보딩 — 일본 의료기관 5곳 담당

진코카이

토조

오우치

노구치

아사리 클리닉

각 기관별 데이터 세팅·서비스 커스터마이징·실시간 이슈 대응 담당 — 그중 1곳은 2025.12 현지 출장으로 오픈 지원

[보안] 회사 보안 정책상 실제 화면 캡처 및 소스코드는 노출하지 않으며, 이후 슬라이드는 구조를 일반화한 개념도로 대체합니다.

온보딩 커스터마이징 & 해외 시연 프로토타입 지원

신규 의료기관 재택 서비스 온보딩

2025.09 – 2026.01

ASP.NET(C#) · DevExpress · Oracle PL/SQL

- 기관별로 다른 업무 흐름에 맞춰 서비스 커스터마이징 및 신규 화면 개발
- 여러 기관의 데이터를 통합 처리하기 위해 DB Link 설계·적용
- 연도·일별 통계 화면 개발 → 기관 운영진 의사결정 데이터 제공
- 오픈 현장 파견 — 실시간 에러 대응 및 데이터 세팅

일본 의료박람회 시연용 프로토타입 개발 지원

2025.04 – 2025.06

.NET(C#) · Oracle PL/SQL · Vue.js

- 하이브리드 웹/앱 프로토타입 개발 지원 — 해외 박람회 시연용
- 진료단계별 실시간 푸시메시지 생성·전송 연동 로직 구현
- 예약·접수 키오스크 UI 개발
- 현지 담당자와 지속적으로 소통하며 상충하는 요구사항을 조율·보완

반복 조회(N+1) 구조 개선 — 조회 성능 최적화

데이터가 늘어날수록 목록 화면의 응답이 느려지는 문제가 있었습니다. 원인은 목록을 조회한 뒤 각 항목마다 상세 정보를 다시 조회하는 N+1 쿼리 구조였습니다.

AS-IS

목록 1회 + 상세 N회 조회

Step 1

목록 조회 API 호출 (1회)

Step 2

목록의 각 행마다 상세 API
반복 호출 (N회)

Step 3

행 수가 늘수록 요청 수·응답
시간이 선형으로 증가

데이터가 많아지는 기관일수록 화면이 눈에 띄게 느려짐 → 사용자 체감 지연 발생

TO-BE

통합 프로시저로 1회 조회

Step 1

목록+상세 조회를 하나의
저장 프로시저로 통합

Step 2

문자열 패턴 조건은
REGEXP_SUBSTR로 처리

Step 3

다건 결과는
SYS_REFCURSOR로 반환, 1
회 호출로 완결

요청 수를 N+1 → 1로 축소, 데이터 규모와 무관하게 안정적인 응답 속도 확보

'되돌릴 수 있는가'를 먼저 확보하는 운영 방식

운영 중인 의료 시스템에서는 배포 실패가 곧 진료 중단입니다. 형상관리가 적용되지 않는 DB 객체와 실 데이터를 다룰 때, 저는 작업 전에 항상 '되돌릴 방법'을 먼저 만들어두는 절차를 스스로 정해 지켰습니다.

DB 객체 이력 관리 부재

문제

프로시저·평션은 Git 등 형상관리 대상이 아니어서, 수정 후 문제가 생기면 이전 로직으로 되돌릴 근거가 남지 않음

대응

수정 전 기존 객체를 백업본으로 먼저 생성하고, 그 위에 변경분을 얹는 절차를 규칙화 — 배포 시점마다 복구 가능한 이전 버전을 항상 확보

실 DB 마이그레이션 되돌리기

문제

운영 DB에 직접 반영하는 마이그레이션은 잘못되면 어떤 행이 이번 작업으로 바뀐 것인지 구분할 수 없음

대응

마이그레이션 대상에 '작업일자 + 작업자 이니셜'을 식별값으로 남겨, 문제 발생 시 해당 배치만 정확히 선별해 롤백할 수 있도록 설계

대용량 처리 중 실패 리스크

문제

데이터량이 많은 기관은 단일 스크립트로 일괄 처리하면 중간 실패 시 어디까지 반영됐는지 파악이 어려워 재실행이 위험

대응

전체를 한 번에 돌리지 않고 일정 건수 단위로 분할 실행 — 실패 지점을 특정할 수 있고, 실패한 구간만 다시 처리하도록 전환

도구가 없다면 절차로 만든다 — 형상관리가 없는 환경에서도 '복구 가능한 상태'를 유지하는 것이 운영 담당자의 책임이라고 생각합니다.

SKILLS

기술 스택 — 프로젝트별 사용 기술

이약머약

프론트엔드 리드 · 서버 구축 · 데이터 증강

Frontend

HTML · CSS · JavaScript ·
Web Share API

Backend

Python · FastAPI ·
SQLAlchemy · PostgreSQL

Infra

Azure VM (Ubuntu) · Nginx ·
HTTPS

AI · Data

Custom Vision · OpenCV ·
albumentations

채워줘, 흠즈!

백엔드 서버 구축 · AI 파이프라인 전담

Frontend

React · Three.js · Azure Static
Web Apps

Backend

Python · FastAPI (Async) ·
SQLAlchemy

AI · LLM

Gemini Robotics-ER · gpt-
image-2

3D · GPU

PyTorch · nvdiffrast · trimesh

Infra

Azure GPU VM · Caddy ·
Speech TTS

ForeShield

대화 도메인 백엔드 · DB 전담

Backend

Python · FastAPI ·
SQLAlchemy (Async)

DB

Azure PostgreSQL · Alembic

실시간

SSE 스트리밍 · 재연결 복구

Infra

Azure Container Apps ·
GitHub Actions

품질

pytest · 통합 테스트 · 회귀 테
스트

이런 개발자입니다

End-to-End 구현력

UI부터 서버, DB, 배포, 데이터 파이프라인까지 서비스
전 구간을 혼자서도 구축

AI 서비스 실전 경험

Vision 모델 학습 데이터 구축부터 LLM API 오케스트
레이션까지 세 번의 실전 프로젝트

검증으로 확인합니다

지표와 실사용의 괴리는 데이터로, 동작하지 않던 방
어는 테스트로 — 원인을 파고들어 해결

사진 한 장으로 알아내는 안전한 복약 가이드

Problem

약 봉투에서 꺼낸 순간 정보가 끊긴다 — 외형이 비슷한 약의 오복용 사고, 운동선수의 도핑 성분 미확인 등 기존 서비스는 모양·각인을 직접 입력해야 해 진입 장벽이 높음

Solution

알약 사진 한 장을 Azure Custom Vision으로 분석해 성분·효능·DUR 안전성·도핑 금지 성분·병용 금기까지 한 번에 안내. 외형(모양·색·제형) 필터 검색, 즐겨찾기·공유 지원

My Role — 프론트엔드 개발 + 서버·데이터 전담

- 프론트엔드 개발 리드 (웹 표준 HTML · CSS · JavaScript)
- Azure VM 단일 서버 아키텍처 직접 구축 (FastAPI · PostgreSQL · Nginx)
- 의약품 데이터 수집·정제·관리 (크롤링 → 선별 → DB 적재)
- 학습용 이미지 증강 파이프라인 구현 (배경 제거 · 회전 · 배경 합성)

90종

식별 대상 의약품 (실용성 기준 선별)

3,773장

자체 구축 학습 데이터셋

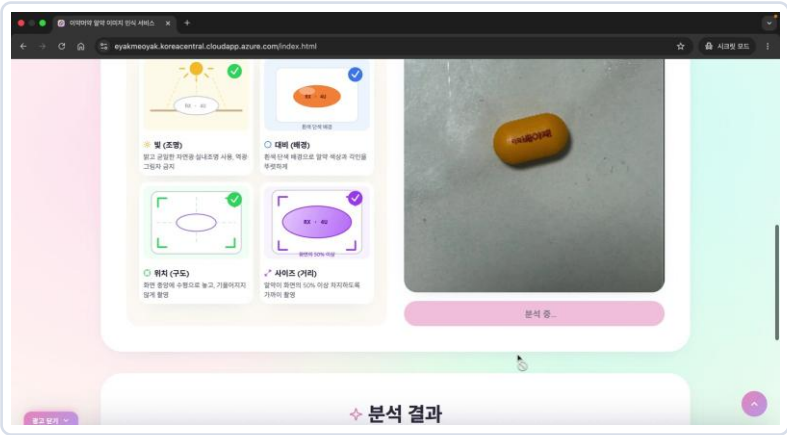
2.4초

AI 분석 평균 응답 시간

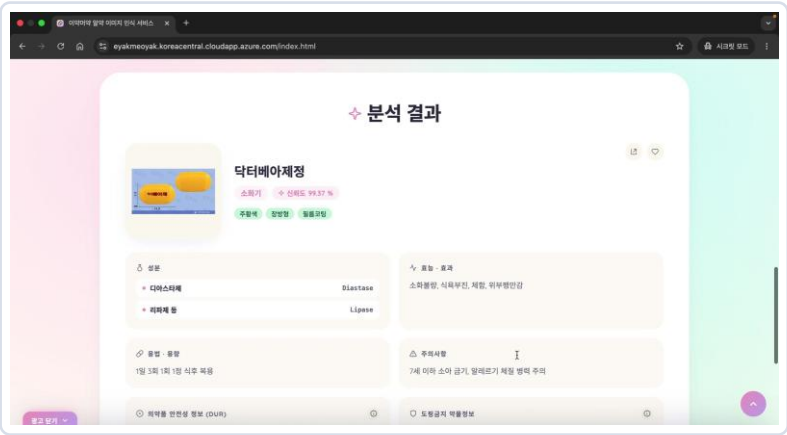
8종 +

제공 정보 (DUR·도핑·병용금기 등)

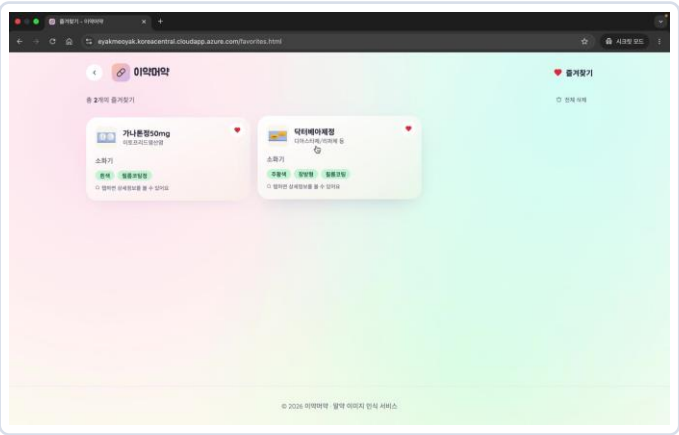
이약머약 주요 화면 — 분석부터 결과·즐거찾기까지



① 알약 사진 업로드 · AI 분석



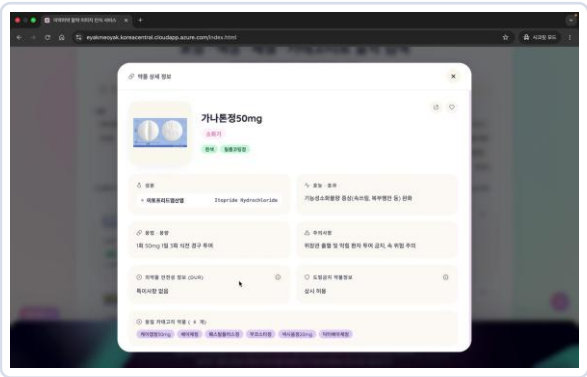
② 분석 결과 — 신뢰도 99.37% · 성분·효능·DUR·도핑



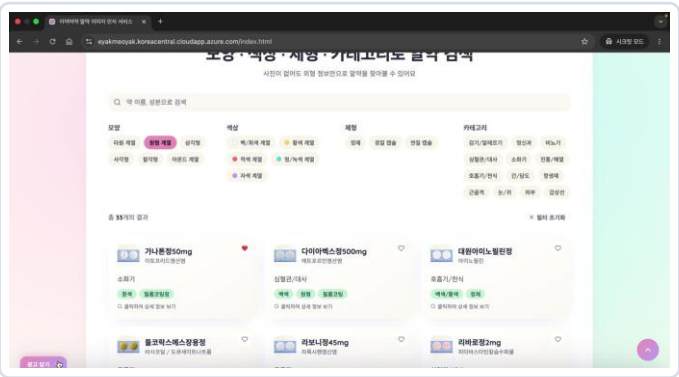
③ 즐거찾기 (localStorage)



메인 화면

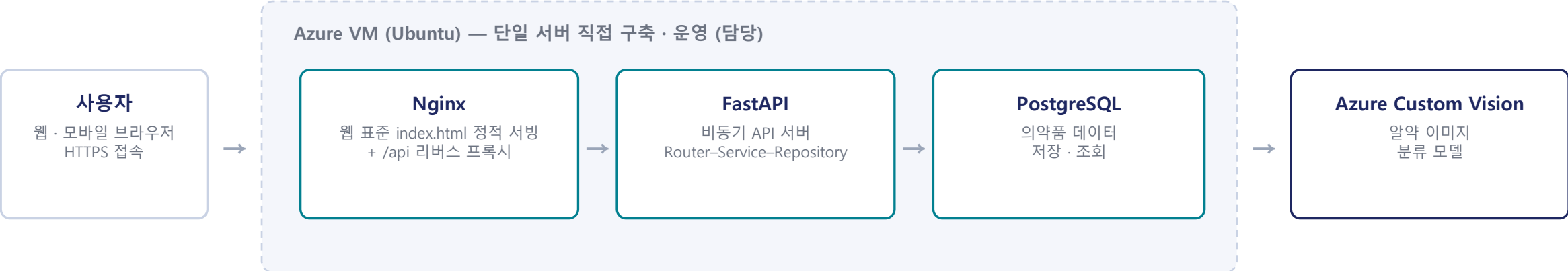


약품 상세 팝업 — 목록·즐거찾기에서 진입

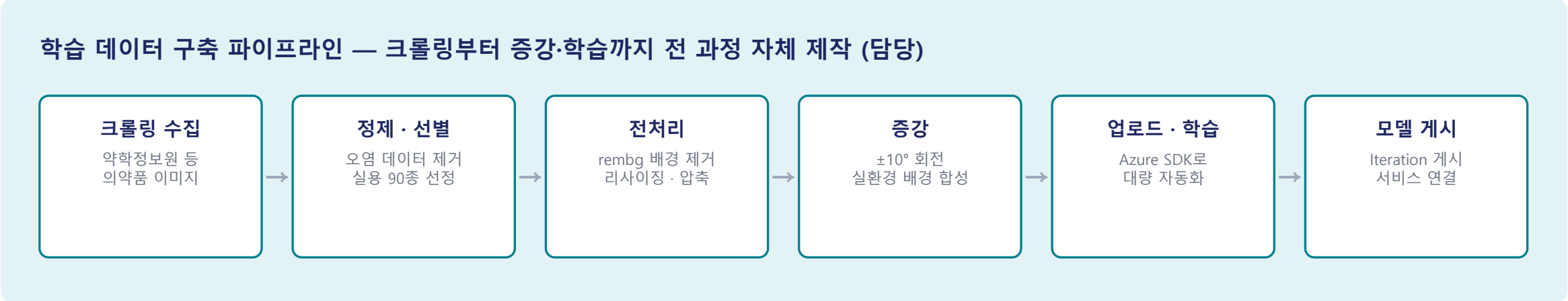


모양·색상·제형·카테고리 필터 검색

단일 VM 기반 All-in-One 아키텍처



프론트엔드(HTML·CSS·JS)와 백엔드를 한 VM에서 운영 — Nginx가 정적 페이지를 서빙하고 /api 요청만 FastAPI로 중계하는 구조로 CORS 문제 없이 단순하게 구성



프론트엔드 — 웹 표준 기반 알약 식별 웹앱

1	웹 표준 UI 구현	프레임워크 없이 HTML·CSS·JavaScript로 알약 식별 UI 전체 구현 (단일 파일 2,300여 줄) — CSS 변수 기반 디자인 시스템, 반응형·고령자 친화 레이아웃
2	AI 분석 플로우	이미지 업로드(PNG·JPG·HEIC) → fetch로 FastAPI /api/drugs 호출 → Azure Custom Vision 분석 결과를 약 정보·안전성 안내와 함께 카드 UI로 렌더링
3	필터 검색	이미지 없이도 모양·색상·제형·카테고리 조합 필터링으로 알약 검색 — 필터 조합으로 정확도 향상
4	즐거찾기 · 공유	로그인 없이 쓰는 localStorage 기반 즐겨찾기(복약 이력 서버 미보관 → 개인정보 보호), Web Share API + OG 메타 태그로 모바일 네이티브 공유
5	배포 · 서빙	백엔드와 같은 Azure VM의 Nginx로 정적 서빙 — /api 경로는 리버스 프록시로 FastAPI 연결, HTTPS 도메인 구성으로 실서비스 공개

HTML5

CSS3

JavaScript (ES6+)

Web Share API

localStorage

Nginx

서버 구축 & 데이터 파이프라인

Azure VM 단일 서버 아키텍처



- Router – Service – Repository 계층 분리로 유지보수성 확보
- HEIC·WEBP 업로드를 서버에서 JPEG 자동 변환 → 아이폰 사용자 지원
- pool_pre_ping · pool_recycle로 DB 커넥션 끊김 장애 예방
- Azure Custom Vision SDK 연동 — 이미지 분류 결과를 확률 순 정렬해 최적 후보 반환
- Nginx 정적 서빙 + 리버스 프록시로 프론트·백엔드 통합 운영

학습 데이터 구축 · 증강 파이프라인

- 1 수집** 약학정보원 등 크롤링 → 처방 통계 기반 실용 90종 선별 (비용·리소스 최적화)
- 2 정제** 단종·구형 디자인 오염 데이터 제거, 수동 검수로 데이터 셋 품질 확보
- 3 전처리** rembg 배경 제거로 객체 중심 학습 유도, 리사이징·압축으로 업로드 제한 대응
- 4 증강** albumentations 회전($\pm 10^\circ$) + 실환경(탁자·손바닥) 배경 합성 증강 → 3,773장 구축
- 5 학습·배포** Azure SDK(Python)로 대량 업로드·학습 자동화, 취약 클래스 맞춤 증강으로 재현율 평준화

데이터 증강 & 서버 이미지 처리

img_aug_rt.py — 배경 제거 + 미세 회전 + 실환경 배경 합성 증강

```
pill_rgba = process_rembg(pill_path)    # 배경 제거 → RGBA
```

```
# 트러블슈팅 반영:  $\pm 45^\circ \rightarrow 0 \sim 10^\circ$  미세 회전만 적용
```

```
transform = A.Compose([
    A.Rotate(
        limit=(0, 10),
        p=1.0,
        border_mode=cv2.BORDER_REPLICATE,
    ),
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
])
```

```
for i in range(1, NUM_AUG + 1):
    augmented = transform(image=temp_img) ['image']
```

```
# 실환경 (탁자·손바닥) 배경과 랜덤 합성
```

```
bg_img = cv2.imread(random.choice(bg_files))
combined = overlay_pill(bg_img.copy(), pill_aug)
```

왜 이렇게 짰나

- rembg로 배경을 지워 모델이 알약 외형·각인에만 집중하도록 유도
- 과도한 회전은 각인 훼손 → 실험으로 $0 \sim 10^\circ$ 가 최적임을 검증
- 배경 합성으로 과적합 해소 — 실환경 인식률 문제를 해결한 핵심 코드

service.py — HEIC · WEBP 자동 변환 (아이폰 대응)

```
def convert_to_jpeg(contents, filename):
    if filename.lower().endswith(
        (".heic", ".webp")):
        image = Image.open(
            io.BytesIO(contents)).convert("RGB")
        buffer = io.BytesIO()
        image.save(buffer, format="JPEG")
        return buffer.getvalue(), ...
    return contents, filename
```

지표는 좋은데 실전에서 못 맞추는 모델, 데이터로 해결

학습 지표와 실제 성능의 괴리

문제

공공 데이터 배경이 4종뿐 → 학습 지표는 높지만 실환경 촬영 이미지에서 인식을 급락 (과적합)

해결

실사용 환경(탁자·손바닥 등) 배경 이미지를 수집해 알약과 합성하는 Background Augmentation 도입 → 재학습 후 실환경에서도 안정적 인식

회전 증강 시 알약 특징 훼손

문제

$\pm 45^\circ$ 회전·노이즈·가림 등 복합 증강 적용 시 각인·형태가 훼손되고 Bounding Box 이탈 발생

해결

회전 범위를 $\pm 10^\circ$ 로 축소하고 과도한 증강을 배제, '회전' 피처에 집중 → 왜곡 없이 다양성 확보, 각인 안정 학습

특정 알약의 낮은 재현율

문제

배경 증강 후에도 이지엔6프로·지르텍정 등 일부 클래스에서 False Negative 지속 발생

해결

Recall 기준치 이하 취약 클래스 7종을 선별해 객체 특징을 강조한 맞춤형 추가 증강 → 클래스 간 재현율 상향 평준화

빈방 사진 한 장으로 완성하는 AI 홈 스타일링

Problem

1인 가구 인테리어 시장은 커지지만 초기 컨설팅 비용, 설치·반품 비용 등 진입 장벽이 높음 — 구매 전 '내 방에 놓으면 어떨까'를 확인할 방법이 없음

Solution

빈방 사진 + 실측 치수(m)만 입력하면 AI가 창문·문·동선을 파악해 실제 판매 가구를 정확한 크기로 자동 배치한 합성 이미지 제공. 3D 뷰어, 배치 이유 음성 설명(TTS), 구매 링크 연결까지

My Role — 백엔드 & 핵심 AI 파이프라인 전담

- 카메라 복원 → LLM 배치 → 3D 렌더·합성 → 소품 데코로 이어지는 4단계 핵심 파이프라인 단독 설계·구현
- FastAPI 백엔드 서버 구축 — REST API 설계, PostgreSQL 가구 DB 연동
- Azure GPU VM 환경에서 딥러닝 모델 서빙 최적화 (앱 시작 시 1회 로드)
- Gemini · GPT 등 멀티 LLM API 오케스트레이션

간편한 입력

사진 1장 + 방 크기·테마·가구 선택만으로 완성

AI 자동 배치

창문·문·동선 파악, 가구 겹침 없는 최적 배치

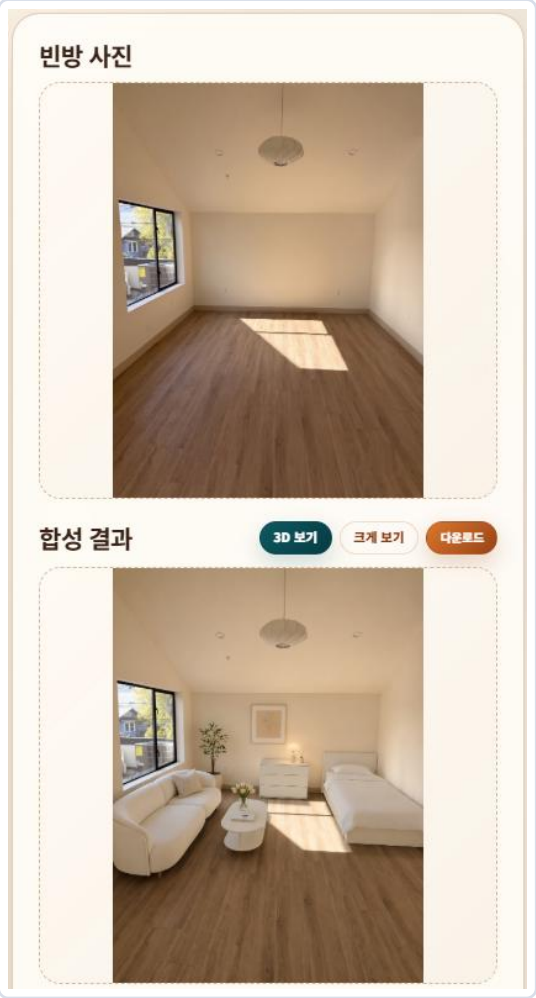
실측 기반 합성

미터 단위 좌표계로 실제 크기 그대로 렌더링

구매까지 연결

배치된 가구의 실제 상품 구매 링크 제공

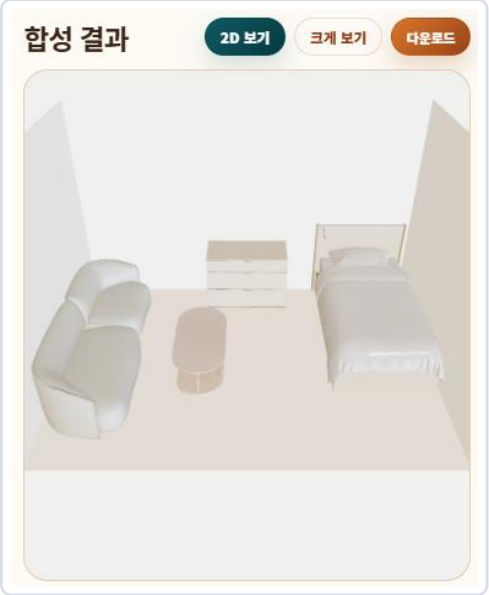
채워줘, 홈즈! 주요 화면



빈방 사진 → 합성 결과



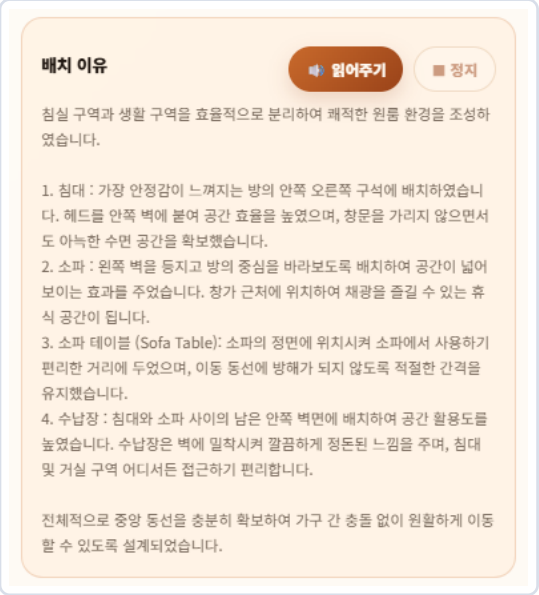
간편한 입력 (치수·테마·가구)



합성 결과 · 2D/크게 보기



사용 가구 · 구매 연결

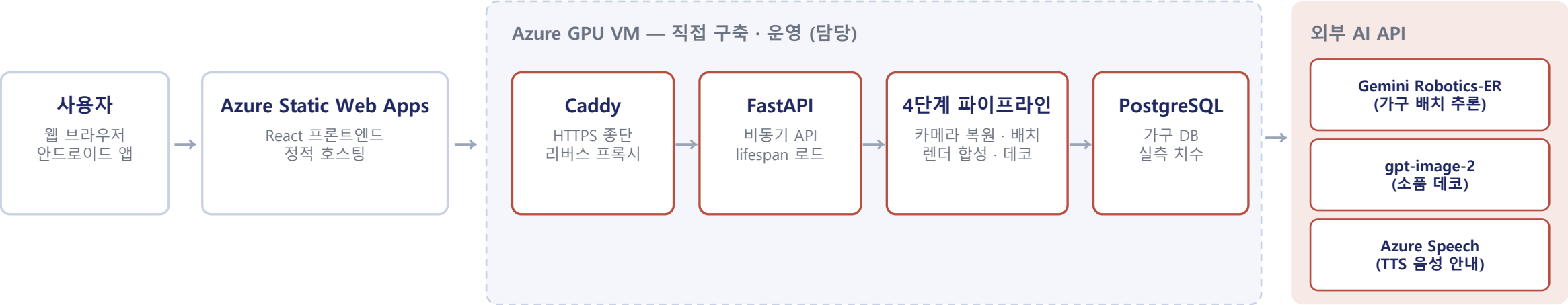


LLM 배치 이유 + TTS 읽어주기

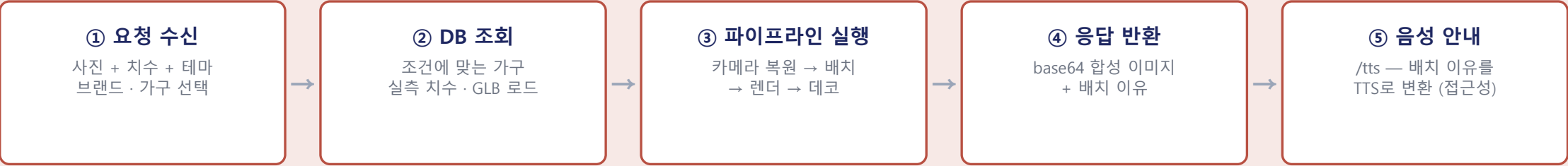


서비스 컨셉 — 빈방을 스타일링

GPU 파이프라인 서빙 아키텍처



요청 처리 흐름 — POST /style-room 한 번으로 전체 파이프라인 오케스트레이션



4단계 AI 파이프라인 — 사진 한 장을 3D 좌표계로

입력: 빈방 사진 + 실측 치수(room_w, room_d) + 미터 스케일 GLB 가구 모델

1

카메라 파라미터 복원

Mask2Former · PerspectiveFields

시맨틱 세그멘테이션으로 바닥면 추출 → vFoV 추정으로 내부행렬 K 구성 → 소실점 기하 + 역투영으로 코너·바닥 법선 계산 → 입력 치수로 카메라 높이 역산, 전 좌표를 미터 단위로 확정

2

LLM 스마트 배치

Gemini Robotics-ER 1.6

사진 + 프롬프트를 멀티모달 추론해 가구별 위치(pos_w, pos_d)·회전각(rot_deg)을 JSON으로 결정 — 좌표 부호·회전 규칙을 프롬프트에 명시해 출력 안정화

3

GPU 렌더 · 그림자 · 합성

nvdiffrast · PyTorch

축 정렬 → 회전 → 3D 배치 → 카메라 투영의 좌표 변환 체인 구현. GPU 래스터라이저로 렌더링, 사진 최고 휘도 영역에서 광원 방향을 추정해 그림자 생성, painter's algorithm으로 원본 배경 보존 합성

4

소품 데코레이션

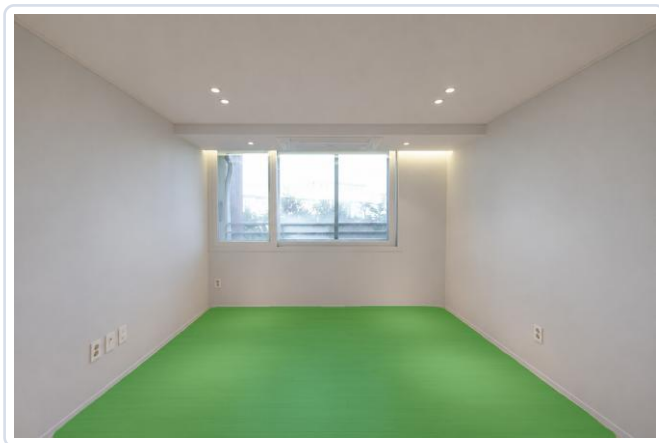
gpt-image-2 (images.edit)

합성 이미지에 램프·화병·액자 등 소품 추가 — 모델의 전체 재생성 특성을 수용하되 '배경·기존 가구 불변' 프롬프트 제약으로 자연스러운 결과 확보

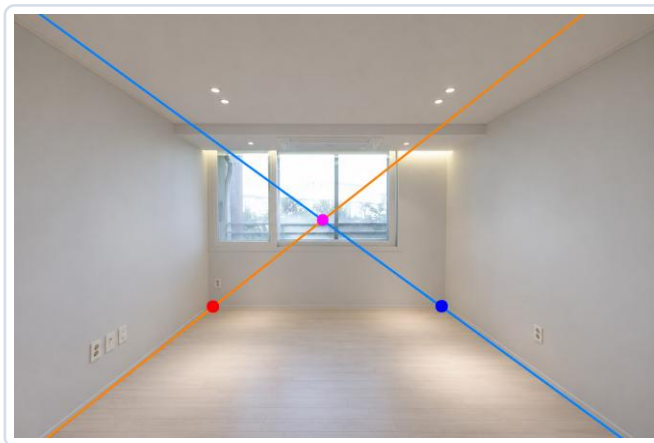
출력: 합성 이미지 + 배치 이유 + Three.js 3D 뷰어 세팅 데이터 · 1회 합성 API 비용 약 54원 수준으로 관리

빈방 사진이 스타일링된 방이 되기까지

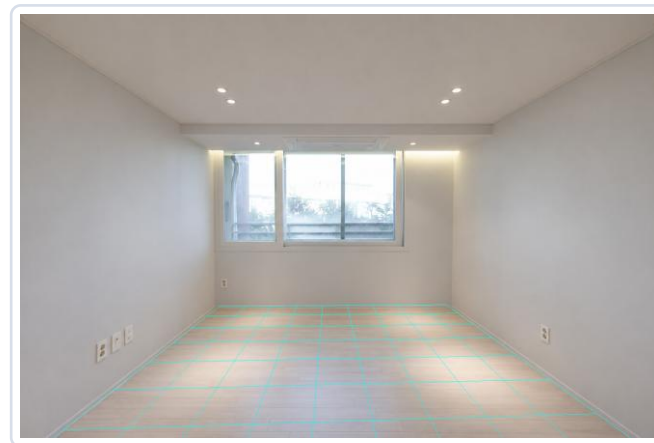
STEP 1 — 카메라 파라미터 복원



① Mask2Former 바닥 검출

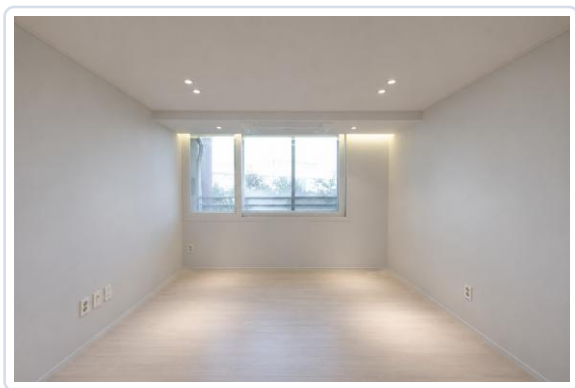


② 소실점 · 코너 기하 계산

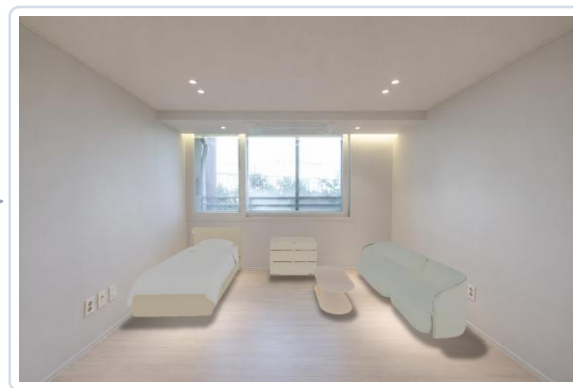


③ 격자 투영 · 미터 스케일 확정

STEP 2~4 — 배치 · 렌더 · 데코



원본 빈방 사진



LLM 배치 + GPU 렌더 · 그림자 합성



gpt-image-2 소품 데코 (최종)

백엔드 서버 설계 — GPU 모델 서빙

FastAPI 서버 · REST API 설계

POST

/style-room — 핵심 스타일링 API

- 입력: 빈방 사진 + 실측 치수(m) + 테마·브랜드·가구 선택
- 처리: DB 가구 조회 → furniture_spec 변환 → 4단계 파이프라인 실행 (요청 한 번으로 전체 오케스트레이션)
- 출력: base64 합성 이미지 + 배치 이유 + 3D 뷰어 데이터
- 예외 처리: 이미지 파싱 실패·빈 선택 등 단계별 HTTPException

POST

/tts — 배치 이유 음성 안내 (접근성 · 책임 AI)

- SSML 구성 → Azure Speech REST를 httpx로 비동기 호출 (ko-KR-SunHiNeural)
- mp3 시그니처 검증 등 방어 로직 후 오디오 스트림 반환

서빙 최적화 · 엔지니어링 포인트

- 무거운 모델(Mask2Former·PerspectiveFields·nvdiffrast CUDA 컨텍스트)을 앱 시작 시 1회만 로드 → 요청당 지연 최소화
- 비동기(async) 요청 처리 + CORS 미들웨어로 웹/앱 클라이언트 동시 지원
- SQLAlchemy ORM 기반 가구 DB 모델링 — 실측 치수(w·d·h)·GLB URL·구매 링크를 한 스키마로 관리
- 환경변수(.env) 기반 시크릿 관리, SSL 필수 DB 연결
- VM에 Caddy를 설치해 HTTPS 종단 직접 구성 — HTTPS 프론트에 서 백엔드 직접 호출 시 발생하는 mixed content 차단 문제 해결

LLM 프롬프트 엔지니어링 — 좌표를 계산하게 만드는 프롬프트

placement_gemini.py — 좌표 규칙을 수식으로 명시한 배치 프롬프트

```
prompt = f"""당신은 원룸 인테리어 배치 전문가입니다.  
빈 방 사진을 보고 가구를 실용적으로 배치하세요.
```

[좌표계]

- pos_w: 0 = 왼쪽 벽, {room_w} = 오른쪽 벽
- pos_d: 0 = 안쪽 벽, -{room_d} = 카메라 앞
- rot_deg: 0=정면, 90=오른쪽 벽, 270=왼쪽 벽

[겹침 금지 - 가장 중요]

- 두 가구 간격 $\geq$ (크기 합의 절반 + 0.6m)
- 배치 후 차지 범위가 겹치는지 반드시 확인

[벽에 붙이기 - 회전에 따라 다름]

- 왼쪽 벽: pos_w = (가구 깊이 $\div$ 2)
- 오른쪽 벽: pos_w = {room_w} - (가구 깊이 $\div$ 2)

```
[출력] JSON: name, pos_w, pos_d, rot_deg"""
```

decorate.py — '금지 조건 우선'으로 생성 모델을 제어한 프롬프트

```
DEFAULT_PROMPT = (  
    "이 방 사진에 인테리어 소품만 추가해줘. "  
    # 불변 조건을 가장 먼저, 단호하게  
    "배경과 가구는 절대 바꾸지 마. 기존 가구, "  
    "벽, 창문, 바닥, 조명, 방 구조와 색감은 "  
    "그대로 유지해. "  
    # 배치 규칙은 번호로 구조화  
    "① 윗면이 평평한 가구 위 중앙에 소품을 "  
    "안정적으로 올려놔. 넘치지 않게. "  
    "② 소품은 카메라에서 잘 보이는 위치에만. "  
    "가구 뒤에 가려지는 곳엔 놓지 마. "  
    "③ 빈 벽에는 액자, 바닥 구석엔 화분이나 "  
    "플로어 조명. 소품끼리 겹치지 않게."  
)
```

방 실측 치수를 f-string으로 주입해 좌표 규칙을 '계산 가능한 수식'으로 제시 → 겹침·벽 이탈을 프롬프트 수준에서 차단하고 JSON 좌표 출력을 안정화

전체 재생성 특성이 있는 이미지 편집 모델에는 '하지 말 것'을 먼저 — 불변 조건 → 번호형 배치 규칙 순서로 서술해 원본 훼손을 최소화

안정 운영 장치 — 쿼터 장애 대응 & 모델 서빙

placement_gemini.py — API 키 로테이션으로 쿼터 장애 자동 복구

```
class GeminiRotator:
    MODEL = 'gemini-robotics-er-1.6-preview'

    def generate_content(self, contents):
        for attempt in range(len(self.api_keys)):
            try:
                model = self._build() # 현재 키로 생성
                return model.generate_content(contents)
            except Exception as e:
                if any(k in str(e).lower() for k in
                    ["quota", "429", "rate limit"]):
                    # 쿼터 소진 → 다음 키로 자동 전환
                    self.idx = (self.idx + 1) \
                        % len(self.api_keys)
                    continue
                raise # 그 외 에러는 즉시 전파
        raise RuntimeError("모든 Gemini 키 소진")
```

main.py — lifespan으로 무거운 모델 1회 로드

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    STATE["camera_models"] = load_models()
    # Mask2Former + PerspectiveFields
    STATE["placement_model"] =
        load_placement_model() # Gemini
    STATE["render_ctx"] =
        load_render_context() # nvdiffrast
    STATE["decorate_client"] =
        load_decorate_client() # gpt-image-2
    yield
    STATE.clear()

app = FastAPI(lifespan=lifespan)
```

수 GB급 모델을 요청마다 다시 올리지 않도록 앱 수명주기에 바인딩 → 첫 로드 이후 지연 없는 GPU 서빙

시연: 트래픽 집중 시 무료 쿼터 소진으로 서비스가 멈추는 문제를 키 7개 풀 + 자동 전환으로 해결 — 쿼터 에러만 선별해 재시도하고 나머지는 즉시 전파

Gemini API

Azure OpenAI

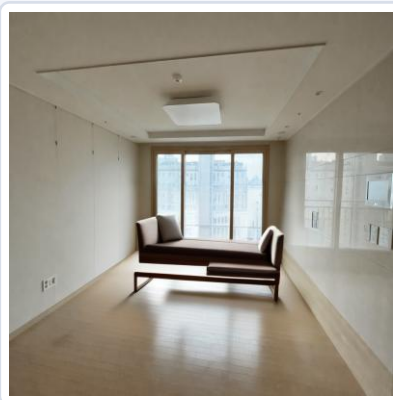
FastAPI

생성형 AI의 한계를 3D 기하로 돌파

Stable Diffusion 합성 품질 문제

문제

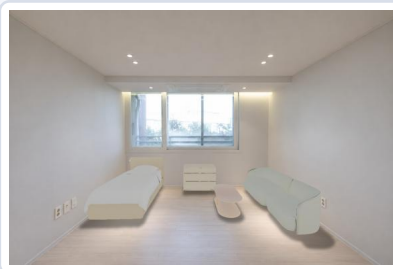
확산 모델로 가구를 합성하자 방 구조가 변형되고 가구가 왜곡 — 원본 공간 보존 실패



SD 합성 — 방 변형 · 가구 왜곡

해결

GLB 3D 모델 기반 실측 렌더링 + 후처리로 전면 전환 → 카메라 복원·GPU 래스터라이징 파이프라인을 직접 구현해 원본 배경을 완전히 보존

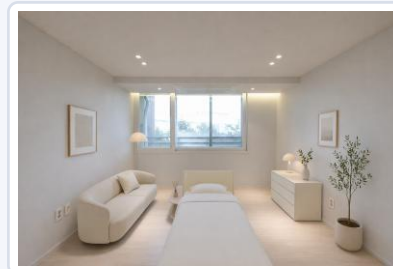


3D 렌더 전환 후 — 원본 보존

GPT-4o + RAG 배치 방식의 한계

문제

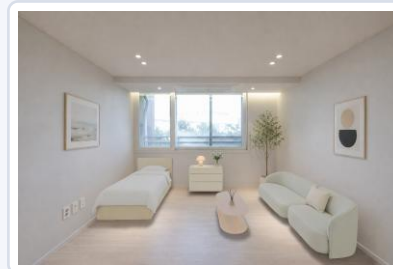
이미지와 가구 실측 치수를 동시에 반영하지 못해 배치 품질 저하 (겹침·비현실적 위치)



GPT-4o + RAG 배치

해결

공간 추론 특화 모델인 Gemini Robotics-ER로 배치 엔진 교체, 좌표 규칙을 프롬프트에 명시 → 겹침 없는 현실적 배치 확보



Gemini Robotics-ER 전환 후

기후재난 6종을 하나의 상황판에서 예측합니다

Problem

재난은 서로 이어지는데 예보는 재난별로 따로 운영됩니다. 가뭄이 산불 예측에 주는 정보가 계산에 들어가지 않고, 담당자는 재난마다 다른 서비스를 각각 확인해야 합니다.

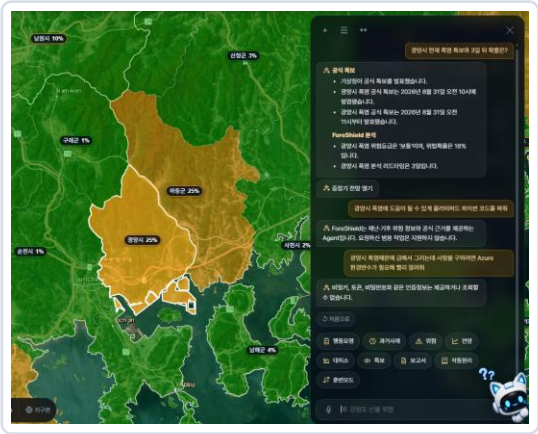
Solution

산불·가뭄·홍수·폭염·한파·태풍 6종을 같은 지역·선행시간 축에 올려 한 화면에서 비교하고, 위험도와 근거·행동요령까지 연결합니다. 판단은 담당자가 합니다.

My Role — 대화 도메인 백엔드 전담

- 대화 도메인 전 계층 단독 구현 (모델 · 저장소 · API · 서비스 · SSE · 요약)
- PostgreSQL 스키마 거버넌스 — 마이그레이션 도구 도입 및 운영 DB 정합성 검증
- 인증 · 세션 보안, 데이터 보존 · 삭제 처리 구현
- 서비스 QA 관리 · 결과보고서 최종 검토

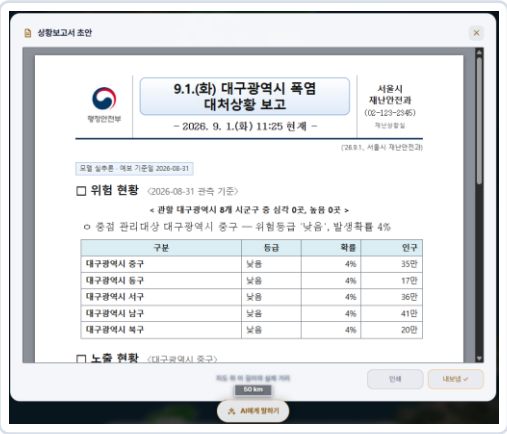
질문하면 답변과 함께 화면이 움직입니다



① AI 어시스턴트 · 지도 연동



② 대한민국 상황판 — 250개 시군구 위험도



③ 상황보고서 자동 생성



④ 지구본 — 전 지구 예측과 재난 기록



⑤ 화면 사용법 안내



⑥ 정부 통합인증 로그인 시안

코드에는 없고 운영 DB에만 있던 것들

DB 스키마를 바꿀 때 코드에 남는 기록이 없었습니다. 운영 DB를 직접 고치면 그 변경은 어디에도 남지 않고, 시간이 지나면 코드가 아는 스키마와 실제 운영 DB가 조용히 갈라집니다.

AS-IS

변경 이력이 남지 않는다

Step 1

필요할 때마다 운영 DB 객체를 직접 수정

Step 2

코드가 아는 스키마와 실제 DB가 조금씩 갈라짐

Step 3

어긋난 사실 자체를 확인할 방법이 없음

운영 DB에만 있는 테이블과 코드와 다른 인덱스가 쌓여 있었습니다

TO-BE

도구로 이력을 남기고 대조한다

Step 1

Alembic 도입, 현재 스키마를 기준으로 생성

Step 2

별도 DB에 기준선을 새로 적용해 스키마 재현

Step 3

운영 DB와 테이블·인덱스 단위로 대조해 일치 확인

이후 스키마 변경은 코드로 남고, 어긋나면 즉시 드러납니다

만든 것을 검증하고, 문서가 제품과 맞는지 확인했습니다

서비스 QA · 결과보고서 검토



- 프론트엔드 26건 · 백엔드 18건 — 담당 영역보다 화면 쪽을 더 많이 검증
- 결과보고서 최종 검토 라운드는 35건 중 26건(74%)을 등록
- 보고서 서술과 실제 제품 표기가 어긋난 곳, 문서 내 수치가 서로 다른 곳을 대조로 확인
- 기술적으로 성립하지 않는 보안 서술을 찾아 삭제 요청

이슈와 CI로 돌아간 개발 프로세스

- 1 이슈 단위 관리**

팀이 GitHub Projects를 PMS로 운영했고, 모든 작업을 이슈로 등록해 Backlog → Todo → In Progress → Review → Done으로 추적했습니다. 백엔드에 이슈 40건을 직접 등록했습니다.
- 2 PR 기반 CI**

main으로 가는 모든 PR에서 린트와 테스트가 자동 실행되고, 통과하지 못하면 병합하지 않는 구조였습니다. 제 PR 40건도 전부 이 관문을 거쳤습니다.
- 3 CI 통합 테스트**

대화-SSE 테스트는 진짜 PostgreSQL이 있어야 돌아갑니다. CI에 일회용 DB 컨테이너를 띄워, 운영 DB를 쓰지 않고도 통합 테스트가 매번 깨끗한 상태에서 돌도록 활성화했습니다.
- 4 CI 복구**

발행 데이터 경로 미설정과 린트 버전 문제로 CI가 멈췄을 때, 기존 위반을 기준으로 정리하고 설정을 보완해 파이프라인을 되살렸습니다.
- 5 검증 기록**

PR마다 무엇을 확인했고 무엇을 일부러 하지 않았는지, 그 이유를 함께 남겼습니다. 나중에 같은 판단을 다시 하지 않기 위해서입니다.

세 번의 프로젝트에서 증명한 것

1

화면과 서버의 언어를 모두 씁니다

이약머약에서 프론트엔드 리드이면서 서버·DB·배포를 전담 — 팀의 기술적 공백을 메우는 역할을 자원해서 맡았습니다.

2

AI를 '서비스'로 만들어 봤습니다

모델 데모가 아니라, 데이터 구축부터 GPU 서빙·비용 설계까지 실제 사용자가 쓸 수 있는 형태로 완성했습니다.

3

문제를 데이터와 구조로 푼다

과적합은 배경 합성 증강으로, 생성 모델의 왜곡은 3D 기하 렌더링으로 — 원인을 파고들어 접근 자체를 바꿔 해결합니다.

4

팀에서 한 도메인을 끝까지 책임집니다

6인 팀에서 대화 도메인을 맡아 설계부터 보안·보존 정책까지 완결했고, 이미 있다고 믿었던 방어가 실제로 동작하는지 검증했습니다.

최성광

sungkwang0908@naver.com | Microsoft AI School 10기

감사합니다